

AgentForge

A Local Multi-Agent AI Software Development Environment

Technical Design Document

Version 1.0 — March 2026

Table of Contents

1. Abstract
2. Vision
3. Core Concept
4. System Overview & Architecture
5. Agent Architecture & Interaction
6. Agent Roles
7. Agent Workflow & Execution
8. Visual Workspace Concept
9. Local AI Execution
10. Real-Time Communication
11. Browser Notification System
12. Storage System
13. Example Execution Scenario
14. Design Principles
15. Technology Stack
16. Future Extensions
17. Conclusion

1. Abstract

AgentForge is a browser-based multi-agent AI system designed to simulate a real software development organization. Instead of relying on a single AI model to complete tasks, AgentForge coordinates multiple specialized AI agents that collaboratively design, develop, test, and document software projects.

The system runs entirely on a local machine and uses lightweight local language models (≤ 7 GB) to enable private, offline AI orchestration.

AgentForge introduces a visual and interactive interface where each AI agent behaves like a team member in a digital workspace. Users can observe real-time collaboration between agents such as a Project Manager,

Frontend Developer, Backend Engineer, DevOps Specialist, QA Tester, and Documentation Writer.

This system demonstrates how multi-agent architectures can emulate real-world engineering workflows while remaining accessible on consumer hardware.

2. Vision

Traditional AI systems operate as a single agent responding to prompts. Complex tasks such as software development naturally require collaboration between multiple specialized roles. AgentForge explores a new paradigm where AI systems replicate the structure of real engineering teams.

Traditional approach:

User → Single AI → Output

AgentForge approach:

User → Project Manager Agent → Distributed Agent Tasks → Integrated
Output

This approach enables:

Capability	Description
Modular Intelligence	Each agent owns a well-defined domain of expertise
Specialized Problem Solving	Agents apply domain-specific reasoning to their tasks
Parallel Task Execution	Independent tasks run concurrently across agents
Traceable Workflows	Every decision, output, and handoff is logged and visible

3. Core Concept

AgentForge functions as a **local AI development company** operating inside the browser. The user provides a prompt such as:

"Build a full-stack portfolio website with authentication."

The system then orchestrates a team of AI agents that collaborate to produce the result. Each agent has:

- A defined role
- A specific task domain
- Communication with other agents
- Visible progress in the UI

The user can observe this collaboration in real time.

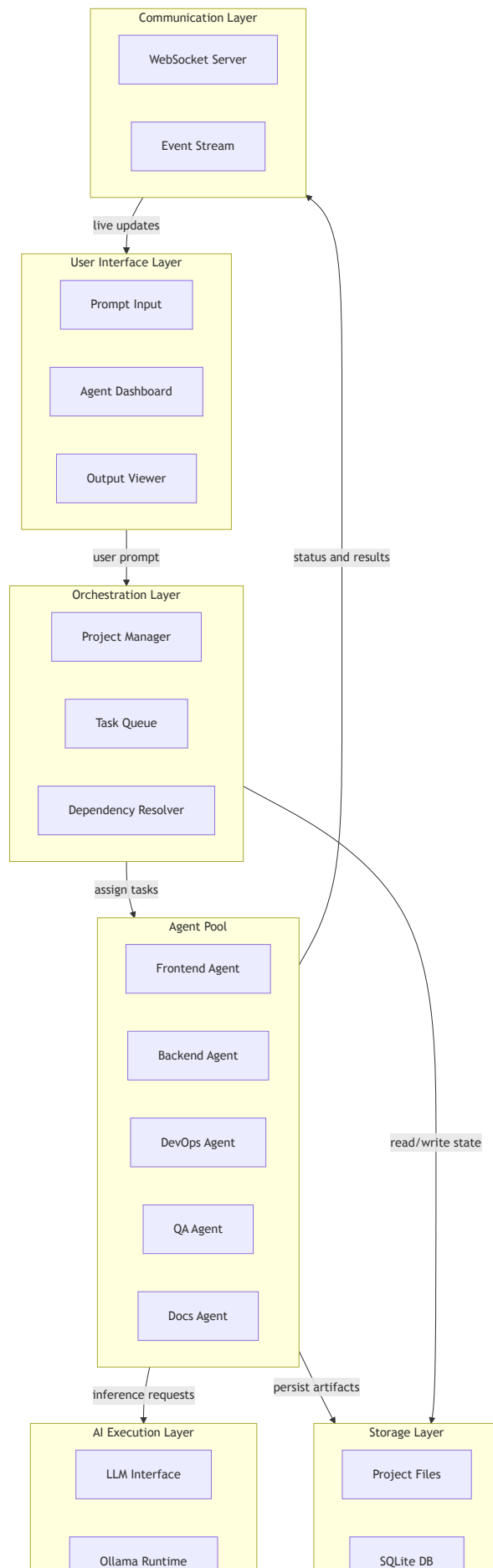
4. System Overview

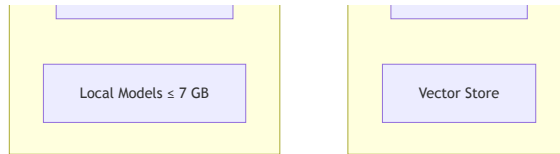
AgentForge consists of five primary subsystems:

#	Subsystem	Purpose
1	User Interface Layer	Browser dashboard for prompt input, monitoring, and output review
2	Agent Orchestration Layer	Central engine that decomposes tasks, assigns work, and manages dependencies
3	AI Execution Layer	Local LLM runtime powering each agent's reasoning
4	Communication Layer	Real-time event streaming between backend and browser
5	Storage Layer	Persistent storage for project files, logs, and agent memory

4.1 High-Level Architecture

Figure 1: AgentForge High-Level Architecture



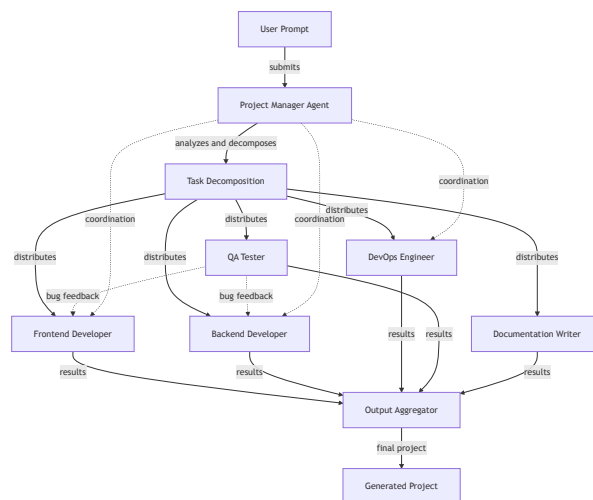


5. Agent Architecture

Each AI agent represents a specialized role in a software development team. Agents operate independently but communicate through a centralized orchestrator. They receive tasks, generate outputs, and report status back.

5.1 Agent Interaction Model

Figure 2: Agent Interaction and Data Flow



6. Agent Roles

6.1 Project Manager Agent

Planning and Coordination

The Project Manager agent acts as the central planning intelligence. It is the first agent to receive the user prompt and the last to sign off on the final output. Responsibilities:

- Analyze the user prompt and identify requirements
- Break the project into actionable tasks
- Assign tasks to specialized agents
- Coordinate progress and resolve dependencies

6.2 Frontend Developer Agent

UI and Client-Side Development

Responsibilities:

- UI/UX design and implementation
- Frontend framework setup (React, Vue, etc.)
- Component generation and layout implementation
- Client-side logic and styling

Typical outputs: React components, CSS styling, frontend project structure.

6.3 Backend Developer Agent

API and Server-Side Development

Responsibilities:

- API route development
- Database schema design and integration
- Authentication systems
- Business logic implementation

Typical outputs: backend services, API routes, database schemas.

6.4 DevOps Agent

Infrastructure and Configuration

Responsibilities:

- Environment setup and containerization
- Project configuration and runtime environment
- Deployment scripts

Typical outputs: Dockerfiles, environment configuration, deployment scripts.

6.5 QA Testing Agent

Quality Assurance and Validation

Responsibilities:

- Error detection and validation testing
- Output verification
- Integration consistency checks

6.6 Documentation Agent

Technical Writing

Responsibilities:

- Generate README files and architecture documentation
- Create setup guides and usage instructions
- Produce API documentation

6.7 Agent Communication Summary

Agent	Receives From	Sends To
Project Manager	User (prompt)	All agents (task assignments)
Frontend Dev	PM (specs), Backend (API contracts)	QA (artifacts), Aggregator
Backend Dev	PM (specs)	Frontend (API contracts), QA, Aggregator
DevOps	PM, Backend	Aggregator
QA Tester	Frontend, Backend (artifacts)	PM (issues), Aggregator
Documentation	All agents (context)	Aggregator

7. Agent Workflow

AgentForge operates using a structured development pipeline with well-defined stages.

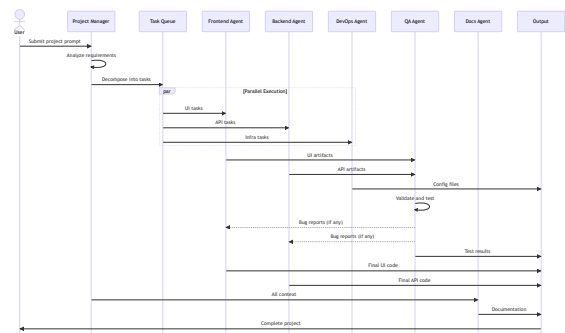
7.1 Task Execution Flow

Figure 3: Linear Execution Pipeline



7.2 Detailed Execution Sequence

Figure 4: Message Passing Sequence



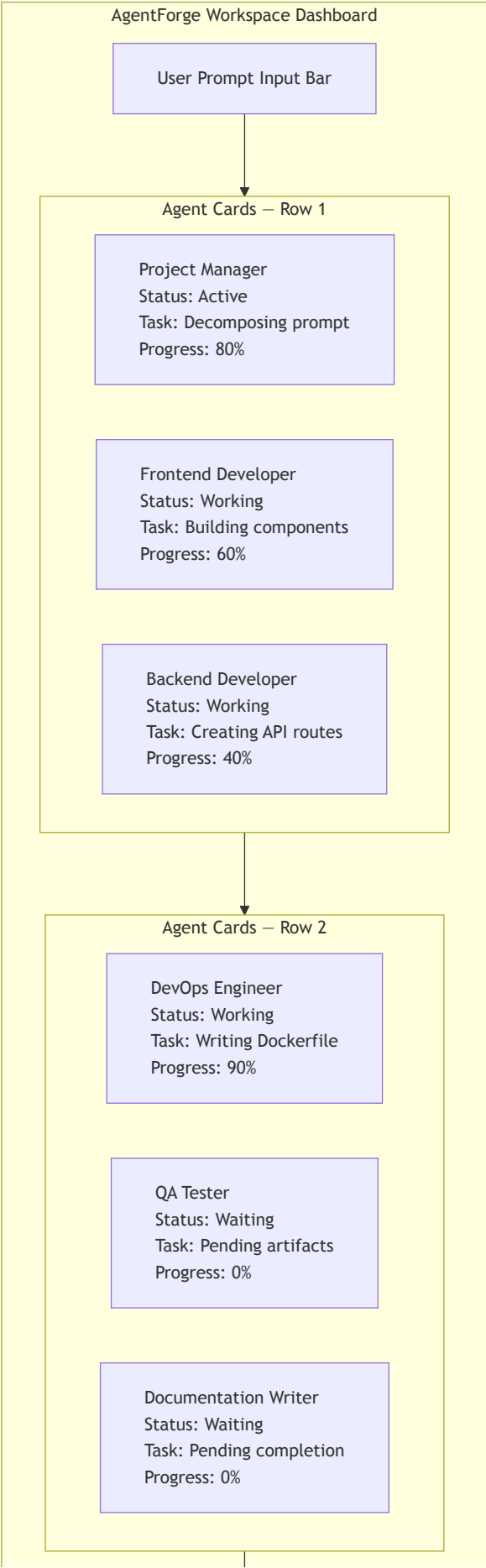
8. Visual Workspace Concept

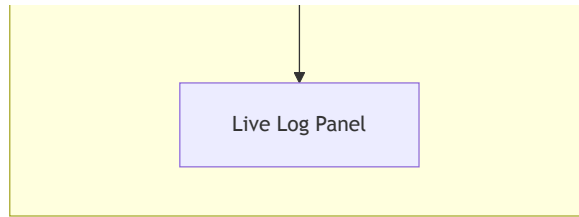
AgentForge includes a visual dashboard that simulates a digital software company workspace. Each agent appears as a card or workspace module that displays:

- **Agent role** — name and specialization
- **Current task** — active assignment from the orchestrator
- **Progress status** — visual completion bar
- **Logs** — real-time output stream
- **Outputs** — generated files and artifacts

The UI updates in real time as agents execute their tasks.

Figure 5: Workspace Dashboard Layout Concept





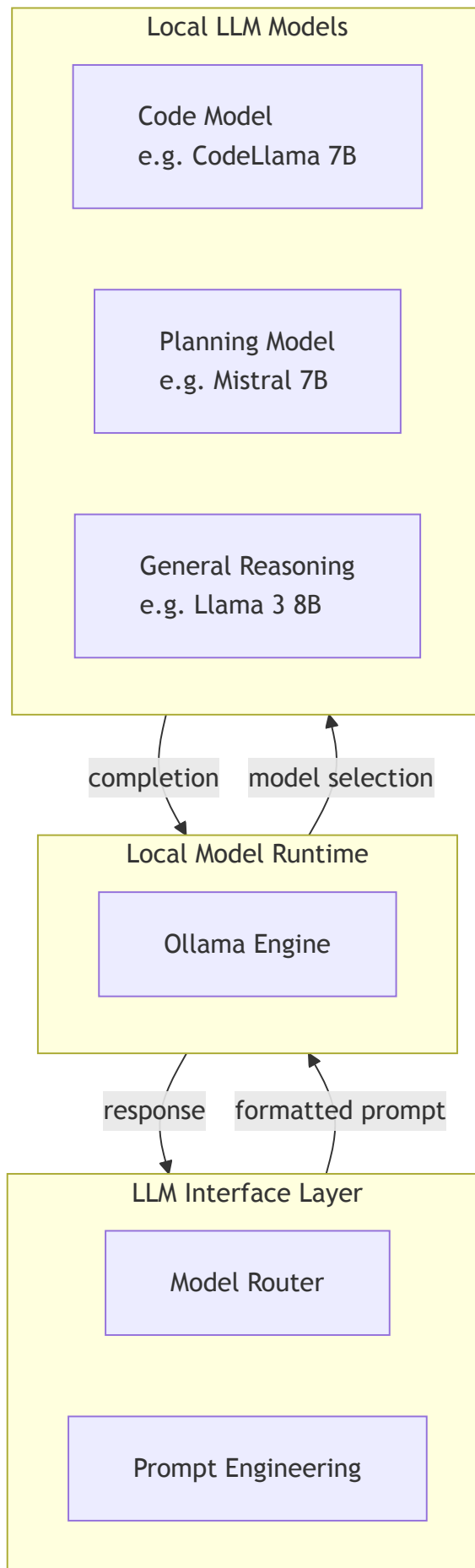
9. Local AI Execution

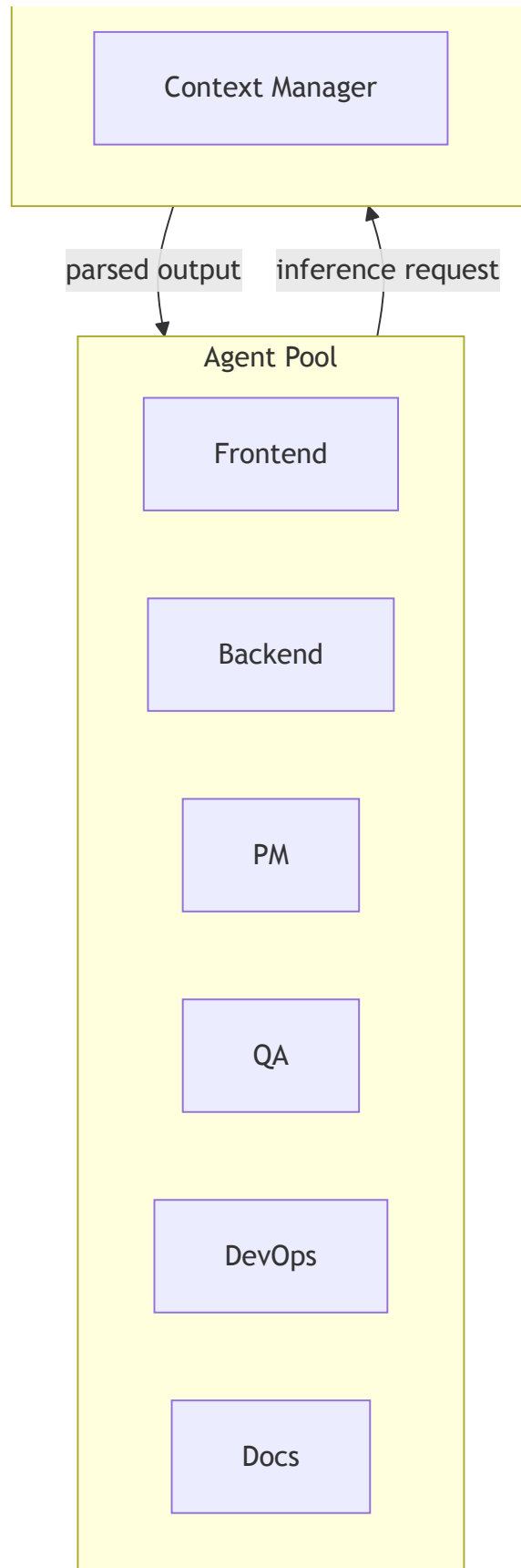
AgentForge runs AI models locally using lightweight LLMs. This ensures:

- **Privacy** — no data leaves the machine
- **Offline operation** — zero internet dependency
- **No API costs** — no external service billing
- **Full user control** — model selection and configuration

9.1 Local Model Architecture

Figure 6: AI Execution Layer





9.2 Model Selection Strategy

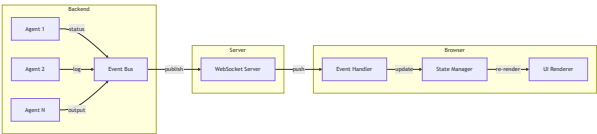
Agent	Recommended Model Type	Reasoning
Project Manager	Planning / Reasoning	Strong decomposition and planning capabilities
Frontend Agent	Code Generation	Optimized for HTML, CSS, JavaScript, React
Backend Agent	Code Generation	Optimized for Python, Node.js, SQL
DevOps Agent	Code Generation	Docker, YAML, shell scripting
QA Agent	General Reasoning	Analysis, comparison, error detection
Documentation Agent	General Reasoning	Natural language generation, technical writing

10. Real-Time Communication System

The system includes real-time updates between backend processes and the browser interface, enabling live agent status updates, task progress visualization, log streaming, and execution monitoring.

10.1 Communication Flow

Figure 7: Real-Time Event Flow



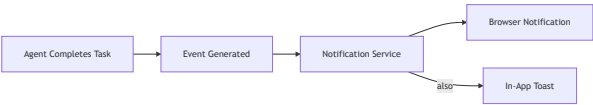
10.2 Event Types

Event	Source	Payload
<code>agent.started</code>	Orchestrator	agentId, taskId, timestamp
<code>agent.progress</code>	Agent	agentId, percentage, message
<code>agent.log</code>	Agent	agentId, level, content
<code>agent.output</code>	Agent	agentId, files, artifacts
<code>agent.completed</code>	Agent	agentId, taskId, duration
<code>agent.error</code>	Agent	agentId, error, stackTrace
<code>project.completed</code>	Orchestrator	projectId, outputPath

11. Browser Notification System

AgentForge includes a local browser notification system. Users receive alerts when key events occur.

Figure 8: Notification Pipeline

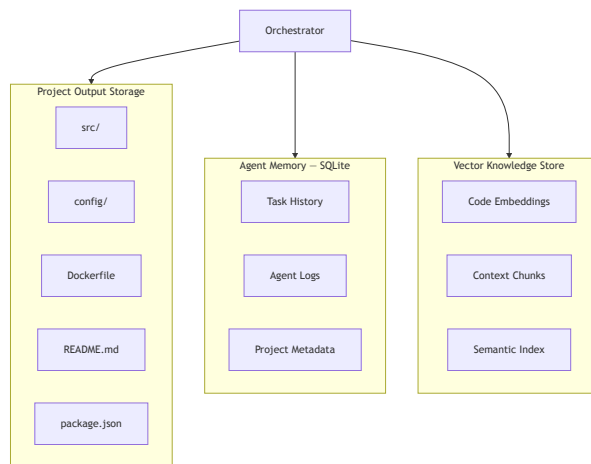


Notification triggers include: task started, task completed, project finished, error detected, and agent waiting on dependency.

12. Storage System

The system stores generated artifacts locally across three storage tiers.

Figure 9: Storage Architecture



Storage Tier	Technology	Purpose
Project Output	Local filesystem	Generated source code, configs, and assets
Agent Memory	SQLite	Task history, logs, metadata, agent state
Vector Store	Google ScaNN	Semantic search over code and context

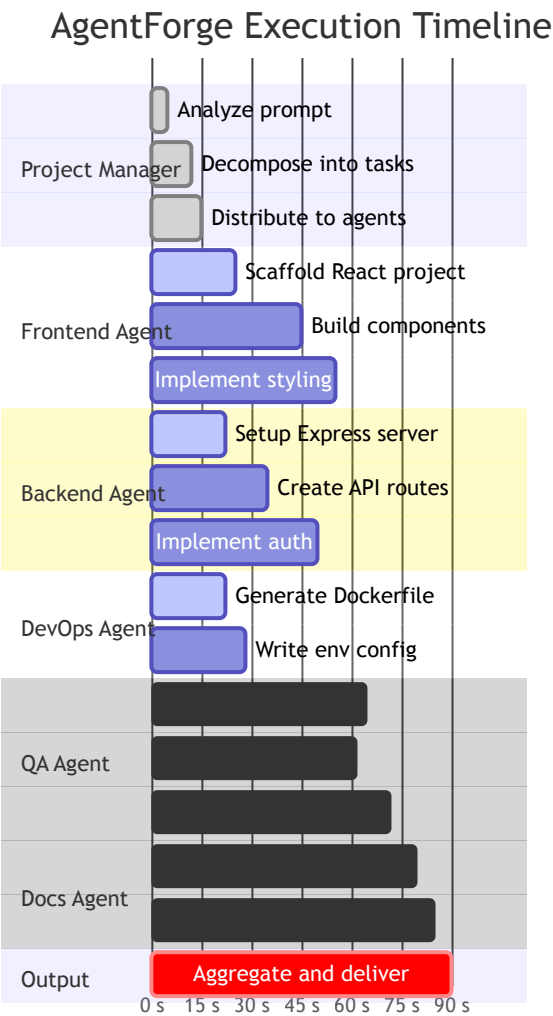
13. Example Execution Scenario

User prompt:

"Create a full-stack portfolio website with authentication."

13.1 Execution Timeline

Figure 10: Gantt Timeline

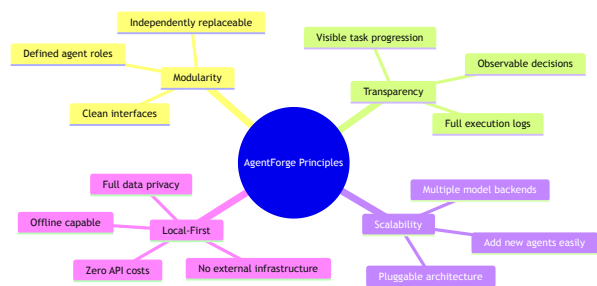


13.2 Step-by-Step Breakdown

Step	Agent	Action	Output
1	Project Manager	Analyze prompt, identify requirements	Task list with 12 subtasks
2	Project Manager	Distribute tasks to agents	Task assignments
3	Frontend Agent	Scaffold React project, build components	src/components/, App.jsx, CSS
4	Backend Agent	Create Express server, API routes, auth	server.js, routes/, models/
5	DevOps Agent	Generate Dockerfile, env config	Dockerfile, .env.example
6	QA Agent	Validate outputs, check integration	Test report, error log
7	Documentation Agent	Generate README, setup guide	README.md, SETUP.md
8	Orchestrator	Aggregate all outputs	Complete project directory

14. Design Principles

Figure 11: Core Design Principles



15. Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	Dashboard UI
Backend	Node.js / Python (FastAPI)	Orchestration and API server
Real-Time	WebSocket (Socket.IO)	Live agent status updates
LLM Runtime	Ollama	Local model inference
Database	SQLite	Task and agent state persistence
Vector Store	Google ScaNN	Semantic context retrieval
Containerization	Docker	Reproducible runtime environment

16. Future Extensions

Extension	Description
Security Analysis Agent	Automated vulnerability scanning and penetration testing
Bug-Fixing Agent	Autonomous detection and repair of code issues
Multi-Model Collaboration	Different LLMs specialized per agent role
Plugin Ecosystem	Community-contributed agent types and workflows
Distributed Agent Networks	Agents running across multiple machines
Analytics Dashboard	Historical performance metrics and optimization insights

17. Conclusion

AgentForge demonstrates how collaborative AI agents can replicate the structure and workflow of real software engineering teams. By combining multi-agent orchestration for parallel and specialized task execution, local AI models for privacy and accessibility, and interactive visualization for transparency and observability, AgentForge provides a new framework for autonomous development systems.

The system highlights the potential of decentralized AI collaboration while remaining accessible on consumer-grade hardware.

AgentForge — Where AI agents build software together.